

제품 개요 및 보안 아키텍처

ODRE PQC v0.2.9

Application-Embedded Post-Quantum Security Boundary

통합 경험 · 보안 경계 · 운영 구조

KOREAN EDITION

2026년 9월

문서 구성

01	1. 제품 개요
02	2. 최소 통합 경험
03	3. 전체 제품 아키텍처
04	4. Fail-Closed 보안 계약
05	5. Platform Adapter와 Native Runtime
06	6. 운영 상태와 명령행 진단
07	7. Worker와 Lifecycle 아키텍처
08	8. 신뢰 경계와 공격 표면
09	9. 배포 구조
10	10. 제품 식별 정보
11	11. 결론

제품 개요 및 보안 아키텍처

Application-Embedded Post-Quantum Security Boundary

ODRE PQC는 애플리케이션 개발자가 양자내성 암호의 세부 구현을 직접 조립하지 않고도, 보호할 요청 경로 앞에 검증 가능한 보안 경계를 설치하도록 설계된 크로스플랫폼 보안 제품이다. 공개 통합 표면은 단순하게 유지하되, 그 내부에서는 Native Runtime의 신뢰성, 암호 Handshake, Session, Sequence, Replay, 요청 의미 결합, 응답 보호, 상태 수명주기와 Fail-Closed 정책을 하나의 제품 계약으로 결합한다.

1. 제품 개요

1.1 ODRE PQC의 정의

ODRE PQC는 PQC 알고리즘 함수만 제공하는 암호 라이브러리가 아니다. 애플리케이션의 보호 경로와 Native PQC Runtime 사이에 위치하여, 보호 요청이 애플리케이션 Handler에 도달하기 전에 필요한 보안 조건을 순서대로 확인하는 Application Security Boundary다.

제품이 담당하는 범위는 다음과 같다.

- 설치된 Runtime과 Artifact의 식별
- ML-KEM-768 기반 Key Encapsulation
- ML-DSA-65 기반 Digital Signature
- Hybrid Handshake와 Session 확립
- Session 만료와 폐기
- Strict Sequence와 Replay 차단
- HTTP Method·Path·Query·Header·Body 의미 결합
- 보호 응답과 보안 Metadata 결합
- 보호할 수 없는 요청의 Fail-Closed 종료
- Worker·Crash·Restart·Shutdown 수명주기 통제
- Windows와 Ubuntu의 Native Trust 차이 흡수
- [doctor](#), [verify](#), [status](#)를 통한 운영 상태 진단

ODRE PQC의 목적은 애플리케이션 전체를 새로운 네트워크 장비나 별도 보안 프록시로 교체하는 것이 아니다. 기존 FastAPI 애플리케이션 안에 명확한 통합 지점을 제공하고, 선택된 보호 경로에 일관된 보안 계약을 적용하는 것이다.

1.2 해결하려는 문제

PQC 알고리즘을 호출할 수 있다는 사실과 실제 애플리케이션 요청을 안전하게 처리할 수 있다는 사실은 같지 않다. 서비스에 PQC를 적용하려면 알고리즘 외에도 다음 문제를 함께 해결해야 한다.

1. 어떤 Native Runtime이 실제로 로드됐는가?
2. 검증한 파일과 실제 로드한 파일이 동일한가?
3. Handshake 결과가 어떤 Session과 요청에 귀속되는가?
4. 동일 요청이나 Sequence가 동시에 재전송되면 Handler는 몇 번 실행되는가?

5. 공격자가 Method·Path·Query·Header를 바꾸어 Handler의 의미를 바꿀 수 있는가?
6. 응답 Body는 보호됐지만 Cookie나 중요 Header가 경계 밖에 남지는 않는가?
7. Worker 충돌이나 Crash 이후 이전 Session·Lease·Status가 재사용되지는 않는가?
8. Runtime의 안전성을 증명할 수 없을 때 요청을 계속 처리할 것인가?

ODRE PQC는 이 문제들을 각각의 선택 옵션으로 남기지 않고, 보호 경로가 Handler로 진입하기 위한 제품 내부 조건으로 묶는다.

1.3 핵심 제품 명제

ODRE PQC의 제품 명제는 세 문장으로 요약된다.

- 통합은 단순해야 한다. 애플리케이션 개발자는 복잡한 암호 상태 기계를 직접 구현하지 않는다.
- 보호 경로의 의미는 끝까지 결합돼야 한다. 암호문 Body만이 아니라 애플리케이션이 실제 처리하는 요청 의미와 응답 의미를 보호한다.
- 안전성을 증명할 수 없으면 실행하지 않는다. 보호 경로는 자동 Classical Downgrade나 조용한 우회를 허용하지 않는다.

1.4 대상 고객과 적용 환경

ODRE PQC는 다음과 같은 환경을 대상으로 한다.

- FastAPI 기반 API 서버
- 기밀성·무결성·재전송 방어가 필요한 보호 API
- 애플리케이션 코드 변경 범위를 제한해야 하는 기존 서비스
- Windows Server와 Ubuntu를 함께 운영하는 조직
- 운영자가 설치 상태와 Runtime 상태를 명령으로 확인해야 하는 환경
- PQC 알고리즘뿐 아니라 요청 처리 경계 전체의 보안 계약이 필요한 환경

공식 검증 조합은 다음과 같다.

항목	검증 조합
Windows	Windows Server 2022 AMD64 / CPython 3.12.10
Linux	Ubuntu 24.04.4 LTS AMD64 / CPython 3.12.3
암호 Runtime	OpenSSL 3.5.8
Key Encapsulation	ML-KEM-768
Digital Signature	ML-DSA-65
Wire Protocol	odre-pqc/3

1.5 기존 보안 구성요소와의 역할 차이

ODRE PQC는 TLS·VPN·WAF를 불필요하게 만드는 제품이 아니다. 각 구성요소는 서로 다른 위치와 책임을 가진다.

구성요소	주된 위치	주된 역할	ODRE PQC와의 차이
TLS	Client와 Server 사이 전송 구간	전송 채널 기밀성·무결성	ODRE PQC는 애플리케이션 보호 경로와 Session·Sequence·요청 의미를 결합한다.

구성요소	주된 위치	주된 역할	ODRE PQC와의 차이
VPN	네트워크 경계	네트워크 접근 범위 제한	ODRE PQC는 허용된 연결 안에서 도 보호 요청의 유효성을 검사한다.
WAF	HTTP 진입부	알려진 공격 패턴과 정책 필터링	ODRE PQC는 암호 Session과 애플리케이션 Handler 실행 조건을 통제한다.
PQC Library	코드 내부	알고리즘 Primitive 제공	ODRE PQC는 Primitive를 요청 처리 수명주기와 Fail-Closed 계약으로 구성한다.
ODRE PQC	애플리케이션 통합 경계	PQC 보호 요청만 Handler로 전달	Runtime Trust부터 Response Binding까지 하나의 실행 경계로 묶는다.

이 구성요소들은 경쟁 관계가 아니라 계층 관계다. ODRE PQC는 기존 전송·네트워크·HTTP 보안 위에 애플리케이션이 실제로 실행되기 위한 보호 조건을 추가한다.

1.6 공개 경로와 보호 경로

모든 경로를 동일하게 차단하는 구조는 실제 서비스 운영에 적합하지 않다. ODRE PQC는 공개 경로와 보호 경로를 분리한다.

- 공개 경로: 서비스 정책에 따라 일반 요청을 계속 처리할 수 있다.
- 보호 경로: PQC Runtime, Handshake, Session, Sequence와 요청 의미 검증을 모두 통과해야 Handler가 실행된다.

보호 Runtime이 초기화되지 않았거나 신뢰 조건이 깨진 경우에도 공개 상태 확인 페이지까지 반드시 중단할 필요는 없다. 반면 보호 경로는 같은 상황에서 우회 실행되지 않는다. 이것이 Fail-Closed with Public Availability 구조다.

2. 최소 통합 경험

2.1 설치에서 실행까지

제품 통합 흐름은 설치·초기화·진단·검증·연결로 나뉜다.

```

Wheel 설치
↓
odre-pqc init
↓
odre-pqc doctor
↓
odre-pqc verify
↓
install(app)
↓
애플리케이션 시작
↓
odre-pqc status

```

각 단계는 다른 목적을 가진다.

단계	목적
설치	Python Package와 명령행 진입점을 배치한다.
init	플랫폼별 Runtime과 로컬 보안 상태를 초기화한다.
doctor	환경·권한·파일·Runtime 전제조건을 진단한다.
verify	실제 PQC Self-test와 설치 완결성을 확인한다.
install(app)	FastAPI 애플리케이션에 보안 경계를 등록한다.
status	실행 중인 상태를 읽기 전용으로 확인한다.

2.2 FastAPI 통합 지점

애플리케이션 측 공개 통합 지점은 다음과 같이 유지된다.

```
from fastapi import FastAPI
from odre_pqc import install

app = FastAPI()
install(app)
```

핵심은 코드 두 줄 자체가 아니라 책임 분리다. 애플리케이션 개발자는 ML-KEM Ciphertext, ML-DSA Signature, Session Store, Replay Window와 Native Provider 경로를 각 Handler에서 관리하지 않는다. 제품이 애플리케이션의 공통 진입 경계에 설치되고, 보호 요청마다 동일한 검증 순서를 적용한다.

2.3 통합 시 애플리케이션이 유지하는 책임

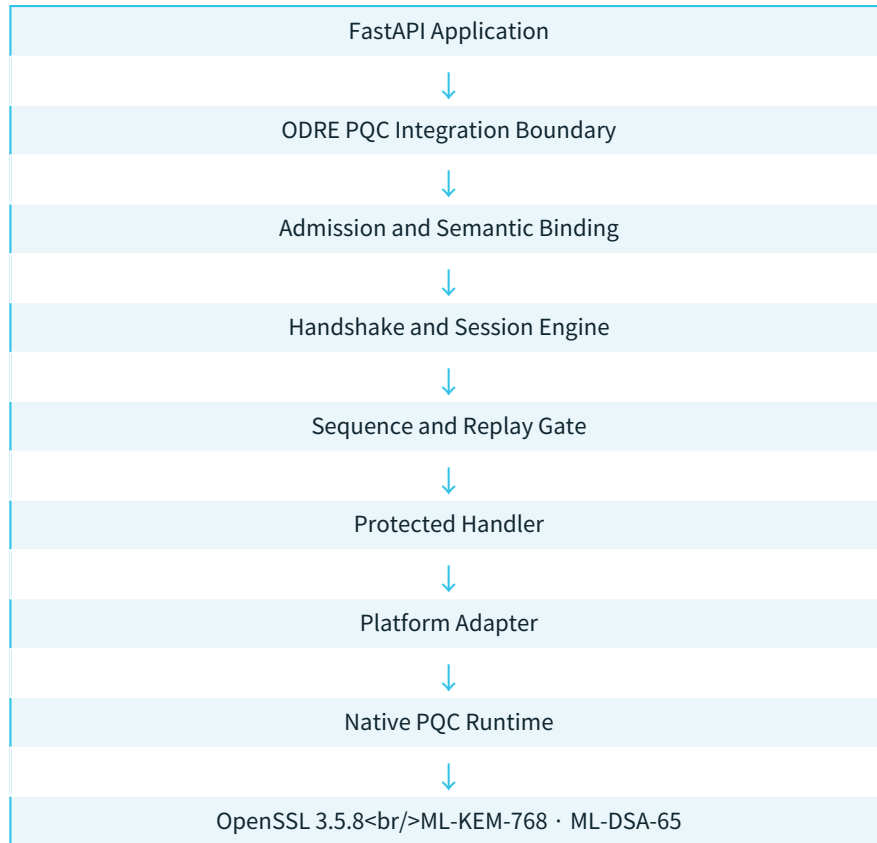
ODRE PQC가 비즈니스 권한 체계나 데이터 분류 정책까지 대신 결정하지는 않는다. 애플리케이션은 다음 책임을 유지한다.

- 어떤 경로를 보호할지 결정
- 인증된 요청 이후의 사용자·조직 권한 확인
- 비즈니스 입력 검증
- 데이터 보존과 감사 정책
- 공개 경로에 허용할 기능 결정
- 오류 응답과 서비스 운영 정책

ODRE PQC는 이 정책이 실행되기 전에 보호 요청이 보안 계약을 통과했는지를 보장하는 역할을 맡는다.

3. 전체 제품 아키텍처

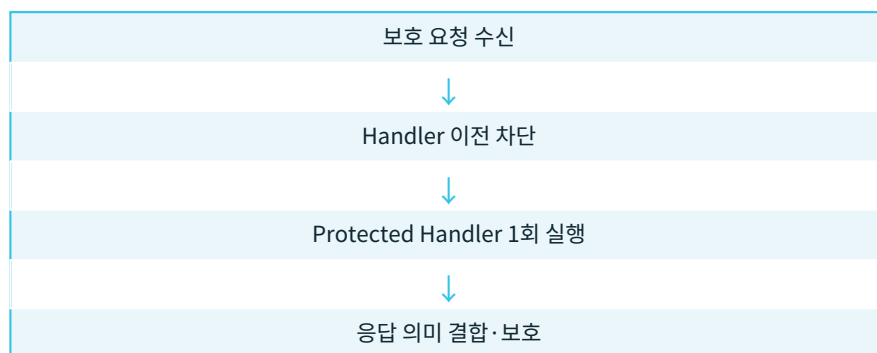
3.1 계층 구조



아키텍처는 크게 여섯 계층으로 구성된다.

1. Application Integration: FastAPI와 보안 경계를 연결한다.
2. Admission: 형식 · 크기 · 지원 유형을 Handler 진입 전에 검사한다.
3. Protocol: Handshake · Session · Sequence · Replay · Envelope를 처리한다.
4. Semantic Binding: 암호학적으로 승인한 의미와 Handler가 처리할 의미를 일치시킨다.
5. Platform Adapter: Windows와 Ubuntu의 파일 · 권한 · Loader 차이를 흡수한다.
6. Native Runtime: 실제 ML-KEM · ML-DSA 연산과 Runtime Identity를 제공한다.

3.2 요청 처리 파이프라인



검증 순서에는 의미가 있다. 입력을 모두 받은 뒤 마지막에 거부하는 방식은 Handler 부작용을 막을 수 없다. 따라서 지원하지 않는 Streaming 유형, 허용 크기를 증명할 수 없는 응답 예약, 잘못된 Envelope는 Handler 이전에 차단한다.

3.3 Admission 계층

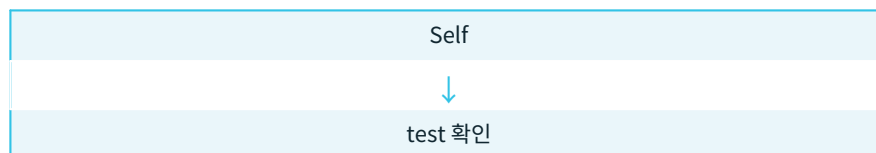
Admission은 암호 연산 전에 공격 표면을 줄이는 계층이다.

- 허용된 Message Type 확인
- 필수 Field 존재 확인
- 길이와 전체 크기 제한
- 비정상 Encoding과 Malformed 구조 거부
- 중복 또는 상충 Field 거부
- 지원하지 않는 Streaming·SSE·Iterator·WebSocket 계약 거부
- 보호 요청에 필요한 Metadata 존재 확인

Admission을 통과했다는 것은 요청이 인증됐다는 뜻이 아니다. 단지 다음 보안 단계가 안전하게 해석할 수 있는 형태라는 뜻이다.

3.4 Handshake와 Session 계층

Handshake는 암호 Primitive를 한 번 호출하는 사건이 아니라, 상대와 Session의 신원을 이후 요청에 결합하는 과정이다.



Handshake에서 결합해야 하는 정보에는 Protocol Version, 알고리즘 조합, 양측 Material, Transcript, 보호 경로 의미와 Session 식별자가 포함된다. 필요한 정보가 일치하지 않으면 Session은 생성되지 않는다.

3.5 ML-KEM-768과 ML-DSA-65의 역할

두 알고리즘은 서로 다른 문제를 해결한다.

- ML-KEM-768: 공유 비밀을 캡슐화하고 복원하는 Key Encapsulation에 사용한다.
- ML-DSA-65: Handshake Material과 Transcript의 출처·무결성을 확인하는 Digital Signature에 사용한다.

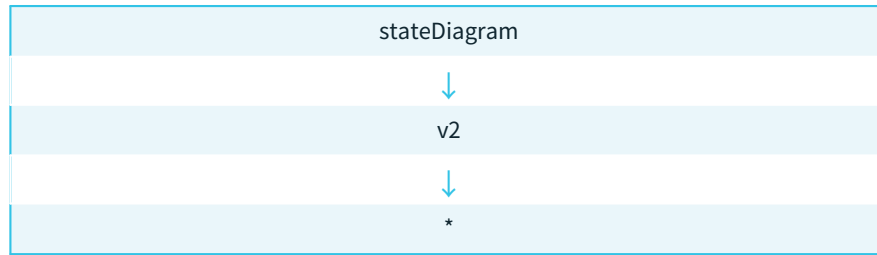
제품은 두 Primitive를 Protocol과 상태 기계 안에서 사용한다. 알고리즘 호출 성공만으로 보호 상태를 선언하지 않으며, Runtime Identity와 Transcript Binding, Session 확립까지 완료돼야 한다.

3.6 Hybrid Handshake

Hybrid 구성은 서로 다른 암호학적 가정과 전환기 호환성을 하나의 Handshake Transcript에 결합하기 위한 구조다. 중요한 점은 여러 알고리즘을 병렬로 호출했다는 사실이 아니라, 결과가 같은 상대·같은 Session·같은 요청 경계에 결합되는지다.

ODRE PQC는 일부 구성요소가 실패했을 때 자동으로 Classical-only 상태를 보호 완료로 간주하지 않는다. 정책이 요구하는 조합을 증명할 수 없으면 Handshake는 실패한다.

3.7 Session 수명주기



Active Session은 무기한 신뢰 객체가 아니다. 만료, Runtime 상태 변화, Worker 소유권 변화, Sequence 위반 또는 무결성 실패가 발생하면 보호 요청에 재사용할 수 없다.

3.8 Strict Sequence와 Replay 방어

Sequence 검증은 단순히 이전에 본 값인지 확인하는 수준이 아니다. 보호 요청은 정확히 다음 Sequence여야 한다.

- Duplicate: 이미 처리한 값이므로 거부
- Rollback: 현재 값보다 작으므로 거부
- Gap: 다음 값을 건너뛰므로 거부
- Race: 동일 Sequence가 동시에 도착해도 하나만 수락

수락 결정과 Sequence 갱신은 Handler 실행과 분리돼 경쟁하지 않도록 원자적으로 처리해야 한다. 그래야 같은 보호 요청이 두 번 도착해도 비즈니스 Handler가 두 번 실행되지 않는다.

3.9 Request Semantic Binding

Body만 암호화하고 외부 HTTP Metadata를 그대로 신뢰하면, 공격자는 암호문을 바꾸지 않고도 Handler의 의미를 바꿀 수 있다. 예를 들어 같은 Body를 다른 Path나 Method에 전달하거나, Query를 변경하거나, 비즈니스 의미에 영향을 주는 Header를 바꿀 수 있다.

ODRE PQC는 다음 요소를 요청 의미에 결합한다.

- HTTP Method
- 정규화된 Path
- 정규화된 Query
- 보안상 중요한 Header
- Payload 또는 Envelope
- Session ID와 Sequence
- Protocol Version과 Message Type

Handler가 보는 값과 암호학적으로 승인된 값이 다르면 보호 요청은 실행되지 않는다.

3.10 Response Binding

응답 보호도 Body에 한정되지 않는다. 보안상 중요한 Status, Header, Cookie와 Metadata가 Body와 별도로 변경될 수 있다면 Client가 해석하는 응답 의미가 달라질 수 있다.

Response Binding은 다음을 같은 보호 결과에 결합한다.

- Status Code
- Content Type

- 보호 대상 Header
- Cookie와 Session 관련 Metadata
- Response Body
- 요청 Session과 Sequence의 대응 관계

응답 계약을 안전하게 구성할 수 없는 유형은 Handler 실행 전 또는 응답 생성 경계에서 명시적으로 거부한다.

4. Fail-Closed 보안 계약

4.1 Fail-Closed의 의미

Fail-Closed는 오류가 발생하면 무조건 프로세스를 종료한다는 뜻이 아니다. 보호 조건을 증명할 수 없는 요청이 보호 Handler로 진입하지 못한다는 뜻이다.

다음 상황에서 보호 경로는 차단된다.

- Native Runtime을 찾을 수 없음
- Runtime Hash 또는 Identity 불일치
- Provider·Dependency·Config 신뢰 실패
- PQC Self-test 실패
- Handshake Transcript 불일치
- Session 없음·만료·손상
- Sequence Duplicate·Rollback·Gap
- 요청 의미 Binding 불일치
- 지원하지 않는 Response Contract
- Worker Lease 또는 상태 소유권 불일치
- 안전한 상태 복구 여부를 증명할 수 없음

4.2 자동 Downgrade 금지

보호 정책이 PQC를 요구하는데 Runtime이나 Handshake가 실패한 경우, Classical-only 연결을 같은 보호 상태로 표시해서는 안 된다. ODRE PQC는 자동 Classical Downgrade를 정상 보호로 취급하지 않는다.

4.3 Handler 실행 전 차단

보안 실패 뒤 HTTP 4xx·5xx를 반환하는 것만으로 충분하지 않다. Handler가 이미 실행했다면 DB 변경이나 외부 호출 같은 부작용이 발생했을 수 있다. 따라서 Admission과 지원 계약 검증은 가능한 한 Handler 이전에 완료한다.

5. Platform Adapter와 Native Runtime

5.1 Cross-platform 설계

Python 통합 API는 같아도 운영체제의 신뢰 Primitive는 다르다. ODRE PQC는 차이를 Platform Adapter 안으로 제한한다.



5.2 Windows Native Trust

Windows 계층은 다음 신뢰 조건을 다룬다.

- DLL과 Provider의 예상 경로
- ACL과 소유권
- Junction · Reparse Point 우회
- Dependency Search Path
- 검증한 파일과 실제 Loaded Module의 동일성
- Runtime 사용 중 교체 시도
- Shutdown 이후 Handle과 Module 수명주기

단순히 로드 전 파일 Hash가 맞았다는 사실만으로 충분하지 않다. 검증 후 교체되거나 다른 Dependency가 로드될 수 있으므로, 실제 Loader가 선택한 Object와 검증 Object의 관계를 확인한다.

5.3 Ubuntu Native Trust

Ubuntu 계층은 다음 조건을 다룬다.

- Shared Object와 Parent Directory 권한
- Symlink와 Unsafe Parent
- FD · Inode ([st_dev](#), [st_ino](#)) Identity
- 검증과 로드 사이 Atomic Replacement
- Provider · Module · Engine · Config Injection
- Loader 환경과 Dependency Resolution
- SIGKILL · Restart 이후 Resource와 상태 복구

경로 문자열은 시간에 따라 다른 Object를 가리킬 수 있다. 따라서 중요한 전환에서는 열린 FD와 Inode Identity를 사용하여 검증 대상과 실제 사용 대상을 결합한다.

5.4 Runtime Archive와 Manifest

플랫폼별 Native Runtime은 Wheel과 분리된 Bundle로 관리될 수 있다. 이때 제품은 다음 관계를 확인한다.

1. 선택한 Platform과 Archive가 일치하는가?
2. Archive Hash가 Manifest와 일치하는가?
3. 추출 결과의 파일 집합과 개별 Hash가 일치하는가?
4. Permission과 Parent Trust가 요구 조건을 충족하는가?
5. 실제 Loaded Object가 검증 Object와 동일한가?
6. Package Version과 Runtime Version이 호환되는가?

5.5 Runtime 초기화

Runtime 초기화는 다음 순서로 진행된다.

```
Platform 식별
→ Manifest 선택
→ Archive · 파일 Hash 확인
→ 안전한 Staging 구성
→ Parent · Object Identity 고정
→ Atomic Promotion
→ Native Load
→ Runtime Version 확인
→ ML-KEM · ML-DSA Self-test
→ 보호 가능 상태 전환
```

필요한 Atomic Primitive나 Identity Proof가 없으면 초기화를 완료하지 않는다.

6. 운영 상태와 명령행 진단

6.1 `init`

`init`은 단순 파일 복사가 아니라 로컬 Runtime과 상태 저장소를 안전하게 준비하는 단계다. 이미 초기화된 위치, 경쟁 초기화, 불완전한 Staging, 신뢰할 수 없는 Parent를 구분해야 한다.

6.2 `doctor`

`doctor`는 운영자가 문제 원인을 찾기 위한 진단 명령이다.

- 지원 OS · Architecture
- Python Version
- Runtime 존재와 Identity
- 파일 · 디렉터리 권한
- Manifest와 Hash
- 필요한 Native Symbol
- Provider와 Dependency 전제조건

- 상태 저장소 접근 가능성

진단은 상태를 임의로 수정하지 않고 관측 중심으로 동작해야 한다.

6.3 verify

verify는 설치가 실제 보호 기능을 수행할 수 있는지 확인한다.

- Package·Runtime Version 일치
- Runtime Load
- ML-KEM-768 Self-test
- ML-DSA-65 Self-test
- Hybrid Handshake
- Session 생성과 폐기
- 보호 요청 왕복
- 자동 Downgrade 없음

6.4 status

status는 실행 상태를 읽기 전용으로 보여준다. 상태 조회 자체가 Session, Lease 또는 보호 상태를 바꾸어서는 안 되며, Secret·Key Material·민감한 내부 경로를 노출해서도 안 된다.

7. Worker와 Lifecycle 아키텍처

7.1 Worker 소유권

여러 Worker가 동일 Runtime이나 상태 저장소를 동시에 소유하려 하면 충돌이 발생할 수 있다. 제품은 지원하는 Worker Model을 명확히 하고, 경쟁 소유권을 조용히 허용하지 않는다.

7.2 정상 종료

정상 종료에서는 다음 순서를 보장해야 한다.

1. 신규 보호 요청 수락 중지
2. 진행 중 요청의 제한된 Drain
3. Session과 Lease 정리
4. Native Resource 종료
5. 상태 저장과 Lock 해제
6. 종료 상태 기록

Native Resource를 먼저 해제하면 진행 중 Handler나 응답 보호가 해제된 객체를 참조할 수 있다.

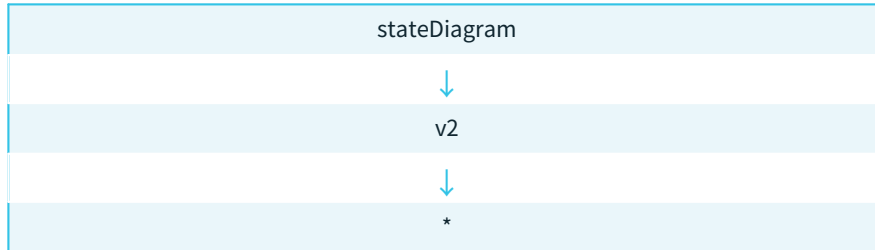
7.3 Crash와 강제 종료

Crash·SIGKILL·강제 전원 종료에서는 정상 Cleanup을 전제로 할 수 없다. 재시작 시 다음을 확인한다.

- 이전 Process와 Worker가 실제로 사라졌는가?
- 남은 Lock이 살아 있는 소유자를 가리키는가?

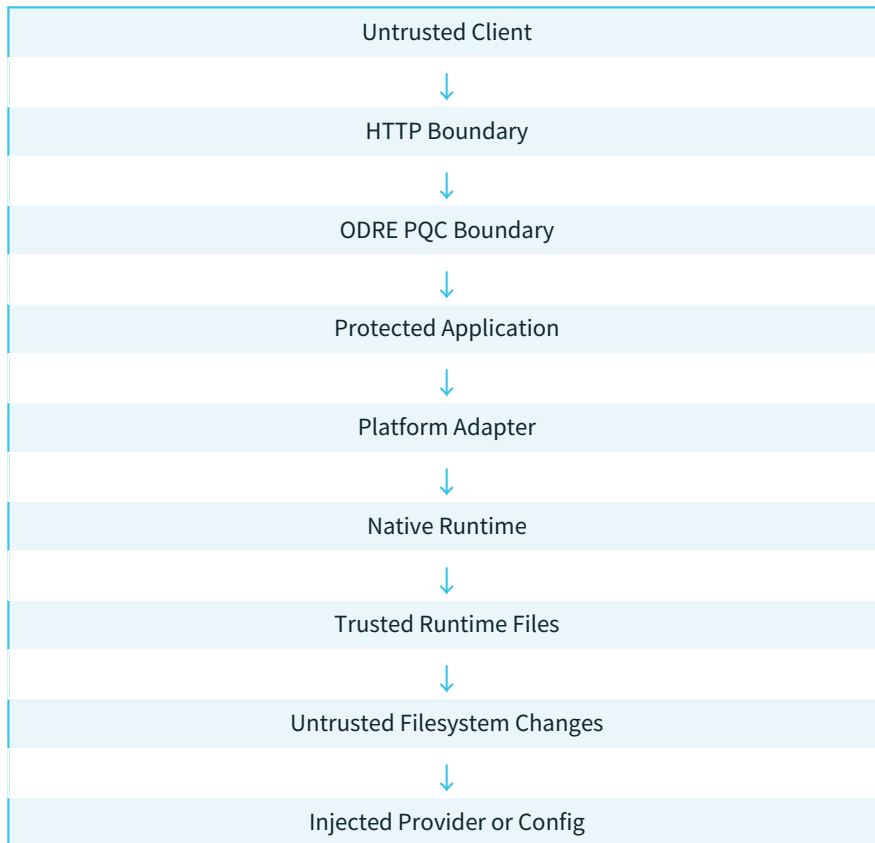
- 이전 Session을 재사용할 수 있는가?
- Stale Lease가 신규 Worker를 방해하는가?
- 이전 Status가 현재 보호 가능 상태를 거짓으로 표시하는가?
- 부분 초기화된 Runtime을 폐기하거나 복구할 수 있는가?

7.4 Restart 상태 전이



8. 신뢰 경계와 공격 표면

8.1 신뢰 경계



주요 신뢰 경계는 다음과 같다.

- 외부 Client와 HTTP Parser 사이
- HTTP Metadata와 인증된 요청 의미 사이
- ODRE PQC와 애플리케이션 Handler 사이

- Python Process와 Native Loader 사이
- 검증 Path와 실제 Filesystem Object 사이
- Worker 상태와 지속 상태 저장소 사이

8.2 주요 공격 표면

공격 표면	대표 공격	제품의 방어 위치
Protocol Admission	Malformed · Oversized · Smuggling Shape	Admission Gate
Session	만료 Session · 교차 Session 재사용	Session Engine
Sequence	Duplicate · Rollback · Gap · Race	Atomic Sequence Gate
Request Meaning	Method · Path · Query · Header 교체	Semantic Binding
Response Meaning	Status · Header · Cookie 변조	Response Binding
Native Files	TOCTOU · Atomic Replacement	Platform Adapter
Loader	Dependency · Provider · Config Injection	Runtime Trust
Filesystem	Symlink · Junction · Reparse · Unsafe Parent	Object Identity
Lifecycle	Stale Lease · Crash · 조기 Shutdown	Lifecycle Controller

8.3 보호 범위 밖의 책임

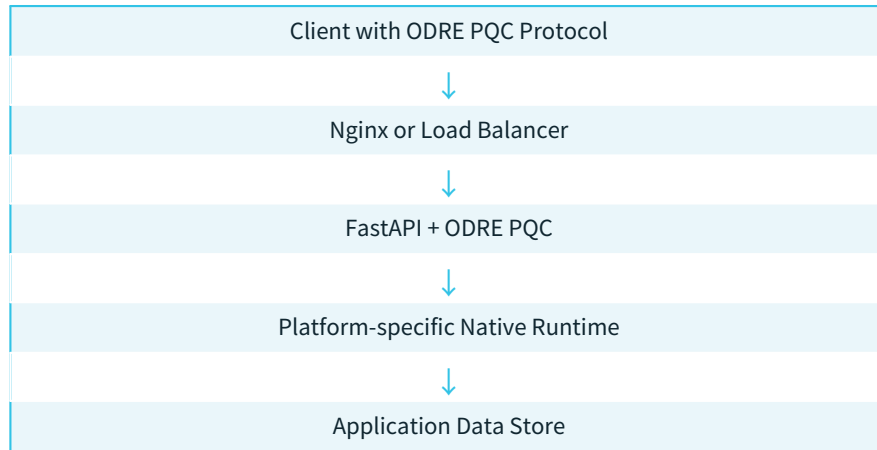
제품 경계를 정확히 이해하려면 포함하지 않는 책임도 명확해야 한다.

- 사용자 계정과 비즈니스 권한 정책 설계
- 애플리케이션 자체의 SQL Injection · XSS 수정
- 호스트 OS 전체 보안 강화
- 네트워크 DDoS 방어
- 물리적 장비 보안
- 데이터 분류와 보존 정책
- Client Device 전체 무결성 보장

ODRE PQC는 이 영역의 보안 제품을 대체하지 않는다. 보호 요청이 애플리케이션 Handler로 진입하는 경계와 그 경계가 의존하는 Runtime 신뢰성을 담당한다.

9. 배포 구조

9.1 기본 배포



Nginx나 Load Balancer는 기존 전송·라우팅 역할을 유지한다. ODRE PQC Security Boundary는 FastAPI 애플리케이션 내부에서 보호 경로가 Handler에 도달하기 전에 실행된다.

9.2 다중 인스턴스 고려사항

다중 인스턴스 환경에서는 다음 항목을 배포 설계에 포함해야 한다.

- Session Affinity 또는 공유 Session 정책
- Sequence 상태의 일관성
- Instance별 Runtime Identity
- Worker Lease와 소유권
- Rolling Restart 중 이전 Session 처리
- Load Balancer Retry가 Replay 의미에 미치는 영향
- Health Check가 공개 경로인지 보호 경로인지 구분

9.3 운영자가 확인할 최소 항목

배포 전 운영자는 다음을 확인한다.

- 공식 지원 OS·Architecture와 일치
- Wheel과 Runtime Bundle의 Version·Hash 일치
- [doctor](#)에서 차단 조건 없음
- [verify](#)에서 실제 PQC Self-test 완료
- 보호 경로와 공개 경로 정책 확인
- 자동 Downgrade 없음
- Restart와 Crash Recovery 절차 확인
- Log와 Status에 Secret이 포함되지 않음

10. 제품 식별 정보

항목	값
제품	ODRE PQC
버전	0.2.9
공식 Wheel	odre_pqc-0.2.9-py3-none-any.whl
Wheel SHA-256	A8AFA10EE0AFACEF1C0FAF55FF07B96C722920E5AEE8692F6F026E804F028697
Windows Runtime SHA-256	826303E72A76B10041D26113F465ADD8960825C71F50722D6792174E73F9D12C
Ubuntu Runtime SHA-256	EFAD16698203371D07F25B429F6731B9C0954EEBF5B58A0024789EF0611A78D1
OpenSSL Runtime	3.5.8
PQC	ML-KEM-768 / ML-DSA-65
Protocol	odre-pqc/3
공식 OS	Windows Server 2022 AMD64 / Ubuntu 24.04 LTS AMD64

11. 결론

ODRE PQC는 양자내성 알고리즘 호출부를 애플리케이션에 추가하는 방식이 아니라, 애플리케이션 Handler가 실행되기 위한 보안 조건을 제품 경계로 정의한다. Runtime Identity, Handshake, Session, Strict Sequence, Replay, Request/Response Binding과 Lifecycle을 같은 계약으로 묶고, 조건을 증명할 수 없는 보호 요청은 Handler 이전에 차단한다.

제품의 외부 사용 경험은 `init`, `doctor`, `verify`, `install(app)`, `status`로 단순화돼 있지만, 내부는 운영체제별 Native Trust와 암호 Protocol 상태 기계를 분리해 처리한다. Windows의 DLL·ACL·Junction·Reparse 특성과 Ubuntu의 SO·Permission·Symlink·FD·Inode 특성을 Platform Adapter에서 흡수함으로써, 애플리케이션에는 동일한 Fail-Closed 보안 계약을 제공한다.

ODRE PQC가 제공하려는 가치는 새로운 알고리즘 이름이 아니다. 개발자가 암호학과 Native Loader의 모든 세부사항을 직접 구현하지 않고도, 보호 요청이 어떤 조건에서 실행되고 어떤 조건에서 차단되는지 설명하고 검증할 수 있는 Application-Embedded Post-Quantum Security Boundary다.